□? Object-Oriented and Software Design Concepts (Simplified)

Abstract (class)

A class you can't create an object from directly. You must make a child class that uses it.

Abstract Data Types

You don't care how it's built — you only care what it can do. Like a remote — you press buttons without knowing the internal circuit.

Access Modifiers

Control who can access parts of your code.

- `public`: Everyone
- `private`: Only the same class
- `protected`: Class + subclasses
- Default (no keyword): Only within the same package

Abstraction

Hiding unnecessary details. Like driving a car without knowing how the engine works.

Attribute

A property or quality of an object. Example: `Student.name`, `Student.grade`.

Behaviours

The actions an object can do. Example: a `Car` can `drive()`, `brake()`, `honk()`.

Encapsulation

Wrapping data + code together and hiding what's not needed outside.

Inheritance

One class inherits features from another. Like a `Dog` inherits from `Animal`.

Polymorphism

One function works differently based on the object. Example: `draw()` works for both `Circle` and `Square`.

Interface Inheritance

A class promises to follow a structure defined in an interface.

Override

When a child class replaces a method from the parent class.

Instantiate

Creating an object from a class.

Getter & Setter

- `getName()` -> Read a private value
- `setName("Sabeer")` -> Set a private value safely

🎁 Structure & Organization

Class Diagram

A visual chart that shows classes and their relationships.

Sequence Diagram

Shows step-by-step interaction between objects.

State Diagram

Shows different states an object can be in, and how it changes.

Unified Modeling Language (UML)

A drawing style to design software visually.

Boundary Object

A middleman between your system and the outside world (like user input/output).

Control Object

An object that manages others, like a project manager.

Entity

A thing in your system, like a `Student`, `Book`, or `Invoice`.

Component

A part of your software that does something specific.

Package

A folder-like structure that keeps related classes together.

Module

A unit of code, like a file or class, that does one thing.

Namespace

A label or container to group things and avoid name clashes.

□ Design Principles & Patterns

Design

The plan of how your software will work.

Design Patterns

Reusable solutions to common problems. Like blueprints for coding.

Decomposition

Breaking a big thing into smaller pieces.

Generalization

Extracting common parts from multiple places to avoid repeating code.

Separation of Concerns

Keep each part of the code focused on one thing only.

Rule of Least Astonishment

Make things work as expected, no surprises!

Liskov Substitution Principle

A child class should work in place of a parent without breaking things.

Conceptual Integrity

The software feels consistent and well thought out, like one person built it.

Service-Oriented Architecture (SOA)

A system built using independent services that can be reused.

□ Code Quality & Maintenance

Code Review

Team checks each other's code to find mistakes and keep style consistent.

Maintainable

Code that is easy to change later.

Maintenance

Fixing or improving software after it’s built.

Reusable

Code that you can use again in different places.

Flexibility

How easily your code can adapt to changes.

Quality Attributes

Things that define how good your software is — like performance, security, etc.

Verification

Making sure your code does what it’s supposed to do.

Requirements

What your software must do (e.g., must support login).

🔄 Advanced Concepts

Coupling

How connected different modules are.

- Tight: Hard to change
- Loose: Easier to manage

Cohesion

How focused a module is.

- High: Clear purpose
- Low: Mixed responsibilities

Degree (Coupling)

Number of connections between modules.

Ease (Coupling)

How clear or hidden those connections are.

Flexibility (Coupling)

Can you swap modules easily?

Deadlock

When processes wait for each other forever, and nothing moves.

Tradeoff

Giving up one thing to gain something else.

Model

A simplified version of a real-world thing.

Model Checking

A process that checks all possible states of the system to find problems.

Counterexample

A case that shows a bug in your logic.

Programming Paradigm

A style of programming — like object-oriented, functional, procedural.

CRC

A way to plan objects by writing down what they do and what other classes they work with.

Information Hiding

Don’t let others see or use parts of your code they don’t need.

Global Variable

A variable that every part of your code can access (not always a good thing).

Local Variable

A variable that is only visible where it’s created.